

Beets is a powerful tool that helps people organize their digital music collections—renaming files, fixing missing information (like album art, artist names, and release years), and keeping everything neatly cataloged. However, beets is a command-line program, and even beyond that, it has configuration files that the user must manually edit.

My project will create a GUI for beets, such that it handles the manual configuration file editing and command running in a simple to use interface. The intent is that the UI remains fairly simple. It can import new music, designate a path for the users music library, and as a stretch goal use plugins.

Core requirements

ID	User Story	Description
1	Launch & Connect to Library	On startup, the program asks the user to create or select a directory to create their library. This in effect, edits the config.yaml file that the user would otherwise have to edit manually. T1 - create general ui - 1 hour T2 - add button to select directory for library - 30 mins T3 - handle config editing in backend - 1 hour
2	Import New Music	Create a button to add new music to the library. This would otherwise be a command the user would run. T1 - create a button to select a directory to add music - 30 mins T2 - handle adding new music to library - 30 mins
3	View Library	Allows the user to see the album, album art and tracks already added to the database, and click on a track or album to see its metadata. (such as release date, artist info, ect.) T1 - Create UI to interact with library - 2 hours T2 - UI interacts with backend - 1 hour

Unfortunately, I have to cut back on the core requirements due to needing to change projects.

Stretch goals

ID	User Story	Description
4	Plugin Management	User can view a list of available beets plugins, toggle them on/off, and have the change written to config.yaml automatically. T1 - build plugin list/toggle UI - 1.5 hr T2 - read/write plugins: block in config - 1 hr
5	Search / Filter Library	User can type into a search box to filter the library view by artist, album, or track title using beets' query syntax under the hood. T1 - add search bar to library view - 30 min T2 - pass query to backend and refresh results - 1 hr

Scope Exclusions

ID	Excluded Item	Description
----	---------------	-------------

X1	Music Playback	The application will not play audio. It is an organizer/manager only, not a media player.
X2	Manual Metadata Editing	The GUI will display metadata (Req. 3) but will not allow the user to hand-edit individual tag fields. Tag correction still relies on beets' automatic matching.
X3	Multi-User / Accounts	No authentication, user profiles, or cloud sync. Single local user assumed.
X4	Online Store / Streaming Integration	No integration with Spotify, Apple Music, download stores, etc.
X5	Full Config Coverage	Only the directory/library path and (stretch) plugins: fields are managed. All other config.yaml options remain manual.
X6	Cross-Platform Packaging	No installer/distributable build is guaranteed. The deliverable runs from source on the development platform.

User stories

Requirement 1 — Launch & Connect to Library

1a. First launch — create a new library (success)

Pre-Condition: The application has never been run; no library path is configured.

Steps:

1. The user launches the application.
2. A first-run dialog appears asking the user to create or select a library directory.
3. The user clicks "Create New Library" and chooses an empty folder.
4. The user confirms.

Expected result: The application writes the chosen path to `config.yaml`, closes the dialog, and opens the main window connected to the new (empty) library.

1b. Launch - select existing library (success)

Pre-Condition: A beets library already exists on disk.

Steps:

1. The user launches the application.
2. The first-run dialog appears.
3. The user clicks "Select Existing Library" and navigates to the existing directory.
4. The user confirms.

Expected result: The config is pointed at the existing library and the main window loads its contents.

1c. Launch — user cancels selection (failure/edge)

Pre-Condition: No library configured; first-run dialog is open.

Steps:

1. The user clicks "Cancel" or closes the dialog without choosing a path.

Expected result: The application does not write a config path. It either re-prompts or exits gracefully with a message stating a library is required to continue. No partial/corrupt config is written.

1d. Launch — invalid/unwritable directory (failure)

Steps: The user selects a directory the application cannot write to.

Expected result: An error message explains the directory is not writable and the dialog remains open for a new selection.

Requirement 2 — Import New Music

2a. Import from a folder (success)

Pre-Condition: Main window open, library connected.

Steps:

1. The user clicks the "Import Music" button.
2. A folder-picker dialog opens.
3. The user selects a folder containing audio files and confirms.

Expected result: The application runs the beets import, the operation completes, and the newly added items appear in the library view (or a "refresh to view" prompt is shown).

2b. Import empty / no audio files (failure/edge)

Steps: The user selects a folder with no recognized audio files.

Expected result: The application reports that no importable music was found; the library is unchanged.

2c. Import cancelled (edge)

Steps: The user opens the folder picker and cancels.

Expected result: No import runs; the user returns to the main window with no change.

Requirement 3 — View Library

3a. Browse albums (success)

Pre-Condition: Library contains at least one album.

Steps:

1. The user opens the main window.
2. The library view lists albums/tracks.

Expected result: All albums and tracks currently in the database are displayed.

3b. View item metadata (success)

Steps:

1. The user clicks an album or track in the list.

Expected result: A detail panel/dialog shows that item's metadata (release date, artist info, etc.).

3c. Empty library (edge)

Pre-Condition: Library is connected but contains no items.

Expected result: The view shows a friendly empty-state message (e.g., "No music yet — click Import Music to get started") rather than a blank or broken screen.

Technical Stories

Req. 1 (T3) — Config editing backend

Pre-Condition: User has selected a library path in the UI.

Behavior: The backend locates the beets `config.yaml` (creating it if absent), sets the `directory:` (and `library:` DB path) key to the chosen path, preserving any existing unrelated keys and valid YAML formatting, then saves. If the file cannot be parsed or written, the backend raises a handled error surfaced to the UI (see Story 1d) and leaves the original file untouched.

Req. 2 (T2) — Import execution backend

Pre-Condition: A valid source folder has been chosen.

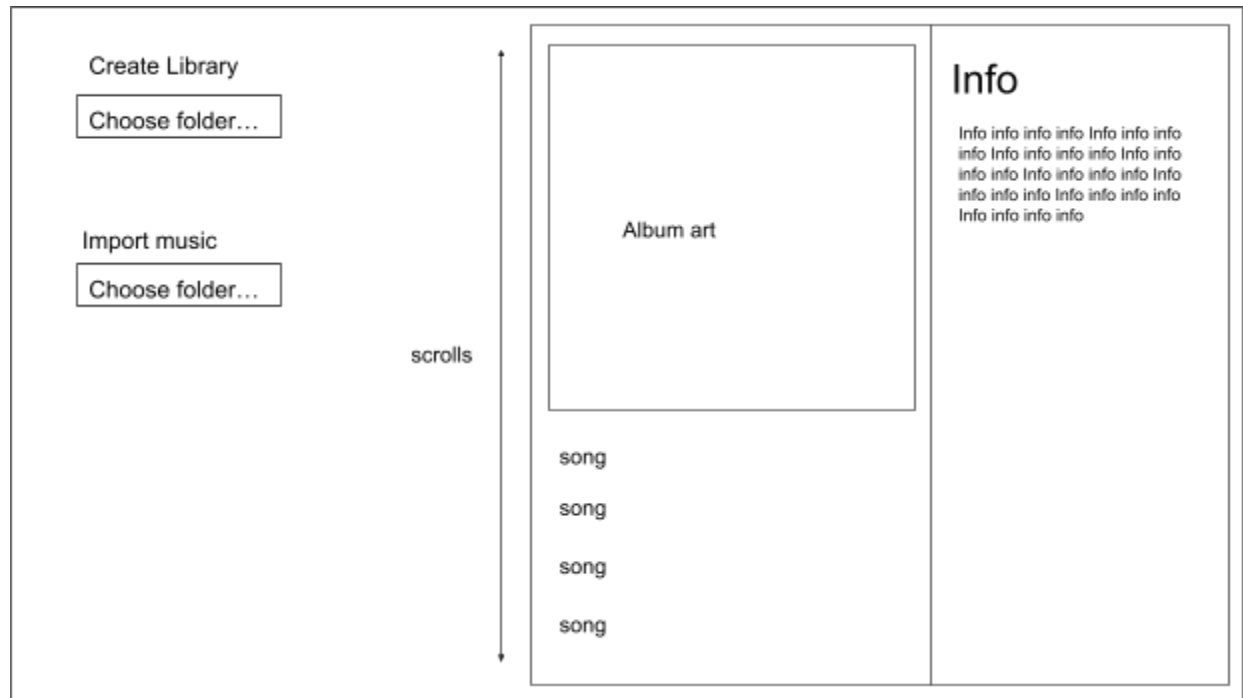
Behavior: The backend invokes the beets import against the selected folder in a mode that does not hang on interactive prompts (e.g., non-interactive/quiet flags). On completion it returns success/failure and item count to the UI. Failures (path missing, no audio, beets error) are captured and returned as a readable message rather than crashing the app.

Req. 3 (T2) — Library query backend

Pre-Condition: Library is connected.

Behavior: The backend queries the beets database for all albums and tracks and returns a structured list (title, artist, album, key metadata) to the UI. Selecting an item triggers a lookup of that item's full metadata. The backend never modifies the database during a read.

Mockup



AI disclosure

I used AI assistance for some parts of this document, specifically the scope exclusions, user stories, and technical stories.